第五章 实验教学详案

本章包含 8 个实验单元，共 16 学时，与项目 4 个 Sprint 节奏严格对应。每个实验单元采用 2 学时连续课时，以"小步骤、可演示、可评审"为设计原则。教师在实验中担任 Coach 角色，巡回指导但不替代学生完成关键决策。

> **实验组织方式：** 4-6 人为一团队，每个团队配置 1 名 PO、1 名 SM、3-4 名 Developer(其中 1 名兼 QA)。角色每个 Sprint 末尾可投票轮换，确保学生体验不同角色。

Lab 1 团队组建与协作工具链（2 学时，第 2 周）

学时	2 学时	周次	第 2 周	Sprint	Sprint 0	地点	实验室

【实验目标】

- 完成 4-6 人的 Scrum 团队组建，明确 PO/SM/Dev 角色分工；
- 建立团队代码仓库、看板、聊天工具与文档库；
- 完成首次 Git 协作：branch、commit、push、Pull Request、merge；
- 产出《团队章程 v1.0》正式版，作为团队治理基础文件。

【环境与前置准备】

- 安装 Git 2.40+、JDK 21、Node.js 20+、Docker Desktop
- 注册 GitHub / Gitee 账号并加入课程组织
- 安装 IDE：IntelliJ IDEA(后端)、VS Code 或 WebStorm(前端)
- 完成 L1 课后的"个人技能矩阵"自评

【实验步骤】

Step 1 团队组建与角色确认(30 分钟)

- ○ 依据技能矩阵互补原则自由组队，4-6 人为一队
- ○ 产生 PO(产品负责人)、SM(Scrum Master)，其余为 Developer
- ○ 每人填写《角色承诺书》，签字提交

Step 2 协作工具链配置(40 分钟)

- ○ Fork 课程组织提供的 campus-overflow-template 仓库到团队组织
- ○ 配置仓库分支保护规则：main 不允许直推、PR 必须经评审
- ○ 在 Gitee 项目 / GitHub Projects 创建看板：To Do / In Progress / In Review / Done
- ○ 建立团队 IM 群(微信/飞书/Discord)用于异步沟通

Step 3 Git 协作首次演练(30 分钟)

- ○ 每位成员 clone 团队仓库，从 main 切出 feature/<姓名拼音>-hello 分支
- ○ 在 docs/team/hello-<name>.md 中写一段自我介绍并提交
- ○ 在 Gitee/GitHub 发起 PR，被一名队友 review + LGTM 后由 SM 合并
- ○ 验收：main 分支应包含全体成员的 hello 文档

Step 4 团队章程撰写(20 分钟)

- ○ 基于课程提供的章程模板，完成本团队版本
- ○ 关键章节：团队价值观、沟通约定、开发规约、DoD 初版
- ○ 提交至 docs/team/charter-v1.0.md，由全员签字(commit message 包含签名)

【提交物】

1. 团队代码仓库 URL(分支保护已配置)；

2. 团队看板截图(包含初始 Backlog 至少 5 条)；

3. 《团队章程 v1.0》Markdown 文档；

4. 每位成员的首次 PR 合并记录截图。

【实验评分要点】

- ● 工具链配置完整性(30%)：仓库、看板、IM、分支保护；
- ● 团队章程质量(30%)：是否具体可执行、是否覆盖关键场景；
- ● Git 协作熟练度(30%)：分支、PR、评审、合并流程是否规范；
- ● 团队氛围与协作情况(10%)：由教师巡回观察评分。

【常见陷阱与排查建议】

- 陷阱：成员各自在 main 上直接提交。规避：第一步即配置分支保护；
- 陷阱：角色仅是名义，SM 不主持会议。规避：本周末 SM 主持首次 Daily Standup，提交录屏或文字纪要。

Lab 2 需求建模工作坊（2 学时，第 3 周）

学时	2 学时	周次	第 3 周	Sprint	Sprint 1	地点	实验室

【实验目标】

- 基于课前的真实用户访谈，产出用户故事地图(User Story Map)；
- 按 INVEST 原则完成 Product Backlog 初版，识别 MVP 切片；
- 使用 Gherkin 语法为高优先级故事编写验收标准；
- 完成 Sprint 1 Planning，产出 Sprint Backlog。

【环境与前置准备】

- 完成 L3 课后访谈作业，每队至少 3 份真实访谈记录
- 准备实体或数字白板：Mural / 飞书白板 / Miro / 钉钉白板
- 阅读 Jeff Patton《用户故事地图》第 1 章

【实验步骤】

Step 1 故事地图共建(45 分钟)

- PO 主导，全员参与，以"用户从未注册到成为活跃贡献者"为骨架
- 横轴：用户活动序列(注册 → 浏览 → 提问 → 回答 → 互动 → 成长)
- 纵轴：每个活动下的具体故事卡片
- 识别 Walking Skeleton(走骨架)切片作为 Sprint 1 MVP

Step 2 Backlog 精化与 INVEST 评估(35 分钟)

- 将地图中的故事卡转化为格式化的用户故事(作为<角色>...)
- 按 INVEST 原则自评每条故事，不通过的现场修改
- PO 按"价值/成本"二维矩阵进行优先级排序

Step 3 Gherkin 验收标准编写(25 分钟)

- ○ 为 Sprint 1 计划承诺的每条故事编写至少 3 个 Given-When-Then 场景
- ○ 区分快乐路径(Happy Path)、边界路径、异常路径
- ○ 提交至 docs/backlog/stories/<story-id>.feature

Step 4 Sprint 1 Planning 会议(15 分钟)

- ○ 基于估算的 Velocity 选择本 Sprint 故事
- ○ 将故事拆分为任务(Task)并分配责任人
- ○ 在看板上完成卡片移入"To Do"列

【提交物】

1. 用户故事地图 PNG / PDF；

2. Product Backlog Markdown 表格(含故事 ID、描述、优先级、Story Point)；

3. 至少 8 条 Gherkin 验收标准文件；

4. Sprint 1 Backlog 截图，任务已分配到人。

【实验评分要点】

- ● 故事地图完整性与故事粒度(25%)；
- ● INVEST 原则达标率(25%)；
- ● Gherkin 标准的可测试性(25%)；
- ● Sprint Backlog 合理性(任务粒度、容量平衡)(25%)。

Lab 3 领域驱动设计事件风暴（2 学时，第 5 周）

学时	2 学时	周次	第 5 周	Sprint	Sprint 1	地点	实验室

【实验目标】

- ● 通过事件风暴(Event Storming)完成 CampusOverflow 领域建模；
- ● 识别核心子域并划定限界上下文边界，产出上下文映射图；
- ● 完成核心聚合识别与不变式提取；

- 产出领域统一语言术语表 v1.0。

【环境与前置准备】

- 准备大量彩色便利贴(橙、蓝、黄、粉、紫)或数字白板
- 所有成员已完成 L7 课后的子域划分初稿
- 准备 1 名"领域专家"角色(可由教师或助教扮演)

【实验步骤】

Step 1 Big Picture 事件风暴(40 分钟)

- ○ 橙色便利贴罗列所有领域事件(过去时：Question Posted、Answer Accepted...)
- ○ 按时间线在长墙上排列事件
- ○ 识别"热点"(Hot Spot)与系统边界事件

Step 2 Process Modeling(30 分钟)

- ○ 为每个事件添加蓝色"命令"卡片(谁触发的)
- ○ 添加黄色"角色"卡片(用户、教师、定时任务)
- ○ 添加粉色"外部系统"卡片(邮件服务、OAuth 提供商)

Step 3 Software Design 与聚合识别(30 分钟)

- ○ 基于命令-事件聚类识别聚合(紫色)
- ○ 验证聚合不变式与一致性边界
- ○ 识别需要的读模型(Read Model)

Step 4 上下文映射与术语表(20 分钟)

- ○ 正式划定限界上下文边界(在墙上画虚线圈)
- ○ 标注上下文间关系：伙伴、客户-供应商、ACL、共享内核
- ○ 提取并提交《统一语言术语表 v1.0》(至少 30 个术语)

【提交物】

1. 事件风暴现场全景照片(墙面 / 数字白板截图)；

2. 上下文映射图(Context Map)PNG；

3. 聚合识别清单 Markdown(含聚合名、根实体、关键不变式)；

4. 统一语言术语表 v1.0(中英对照)。

【实验评分要点】

- 事件识别完整性(25%)；
- 上下文划分合理性(30%)；
- 聚合边界与不变式定义清晰度(25%)；
- 统一语言一致性与团队全员认同度(20%)。

【常见陷阱与排查建议】

- 陷阱：事件粒度过粗(如"用户操作完成")。规避：坚持使用业务术语命名，不接受抽象事件；
- 陷阱：上下文按技术分(数据库上下文、前端上下文)。规避：严格按业务能力划分。

Lab 4 后端工程骨架与领域代码（2 学时，第 7 周）

学时	2 学时	周次	第 7 周	Sprint	Sprint 2	地点	实验室

【实验目标】

- 搭建 Spring Boot + Maven 多模块工程，按 DDD 限界上下文物理分包；
- 完成核心聚合(User、Question、Answer)的 Java 代码骨架；
- 配置数据库连接、Flyway 迁移、统一异常处理与日志框架；
- 产出第一个端到端可访问的接口：POST /api/v1/questions。

【环境与前置准备】

- Lab 3 产出物已提交并通过教师评审
- 本地已安装 MySQL 8 / Docker MySQL 容器
- 团队已完成 L7-L8 的课后骨架代码作业

【实验步骤】

Step 1 Maven 多模块骨架(25 分钟)

- ○ 父 POM 统一依赖版本(Spring Boot 3.x、JDK 21、JUnit 5)
- ○ 建立 co-bootstrap、co-shared-kernel、co-identity、co-qa、co-reputation、co-discovery 六个模块
- ○ 每个子域模块内部分 domain / application / infrastructure / interface 四层
- ○ co-bootstrap 中装配各 context，声明启动入口

Step 2 核心聚合代码实现(35 分钟)

- ○ 在 co-qa-domain 中编写 Question 聚合根、Answer 实体、Vote 值对象
- ○ 所有 Setter 私有化，通过工厂方法或聚合方法保护不变式
- ○ QuestionRepository 接口定义在 domain 层，实现在 infrastructure 层
- ○ 提交首批单元测试覆盖聚合行为

Step 3 基础设施层接入(35 分钟)

- ○ JPA Entity 与领域实体的映射方案(选择：同一个类共用 vs 独立映射)
- ○ Flyway 脚本 V1__init_schema.sql 创建 question、answer、user 表
- ○ 全局异常处理器 @RestControllerAdvice 统一错误响应
- ○ Logback 结构化日志配置(JSON 格式)

Step 4 首个端到端接口贯通(15 分钟)

- ○ 编写 QuestionController.create() 接收请求 → 调用 QuestionApplicationService
- ○ 通过 Postman / HTTPie 验证完整调用链
- ○ 检查数据库实际写入

【提交物】

1. 可运行的多模块 Maven 工程(mvn clean install 全绿)；

2. 通过 application.yml 即可启动并连接到数据库；

3. docs/adr/ADR-002-package-structure.md 记录分模块决策；

4. Postman Collection 包含 POST /api/v1/questions 成功用例。

【实验评分要点】

- 工程结构是否符合 DDD 分层与限界上下文物理分离(30%);
- 聚合实现是否保护不变式(避免贫血模型)(25%);
- 端到端接口可用性(25%);
- 代码风格与文档完整性(20%)。

Lab 5 前端工程骨架与对接（2 学时，第 9 周）

学时	2 学时	周次	第 9 周	Sprint	Sprint 2	地点	实验室

【实验目标】

- 搭建 React 18 + TypeScript 5 + Vite 工程，启用严格类型模式;
- 配置 ESLint + Prettier + Husky pre-commit 钩子;
- 完成路由、状态管理、API 客户端、组件库的基础设施搭建;
- 完成与后端的首个端到端贯通：在前端发起提问并显示。

【环境与前置准备】

- Lab 4 已完成，后端在本地可启动
- Node.js 20+ 已安装，推荐使用 pnpm 包管理器
- 团队已确定 UI 库选择(Ant Design 5 或 Tailwind + 自研)

【实验步骤】

Step 1 工程初始化(20 分钟)

- pnpm create vite@latest 选择 react-ts 模板
- 启用 strict mode 与 noUncheckedIndexedAccess
- 安装核心依赖：react-router-dom v6、zustand、@tanstack/react-query、axios、antd
- 建立目录结构：features/、shared/、pages/、api/

Step 2 代码质量基础设施(20 分钟)

- ○ ESLint + @typescript-eslint + eslint-plugin-react-hooks
- ○ Prettier 与 EditorConfig 统一格式
- ○ Husky + lint-staged 配置 pre-commit 钩子
- ○ 在 vite.config.ts 配置 path alias(@/...)

Step 3 API 客户端生成与封装(30 分钟)

- ○ 基于后端 openapi.yaml 用 openapi-typescript-codegen 生成 TS 类型与请求函数
- ○ 封装 axios 拦截器：Token 注入、错误统一处理、Loading 全局状态
- ○ 建立 useQueryClient 全局配置与 ErrorBoundary

Step 4 首页与发布问题页面(40 分钟)

- ○ AppShell 布局：Header + Sidebar + Outlet
- ○ Routes：/questions、/questions/new、/questions/：id
- ○ 函数组件 + Hooks 实现 QuestionListPage 与 NewQuestionPage
- ○ 使用 TanStack Query 实现请求与缓存
- ○ 提交问题成功后跳转详情页，验证端到端流程

【提交物】

1. 前端工程 pnpm dev 可启动，首页可访问；

2. pnpm lint 与 pnpm typecheck 全绿；

3. pre-commit 钩子可拦截不合规提交；

4. 提问 → 列表显示 → 详情查看 端到端跑通，录屏 30 秒以内。

【实验评分要点】

- ● 工程结构与命名规范(20%)；
- ● TypeScript 严格模式无 any(25%)；
- ● 函数式 + Hooks 范式正确使用(25%)；
- ● 端到端串联可用性(30%)。

【常见陷阱与排查建议】

- 陷阱：大量 useState 散落组件内导致 prop drilling。规避：严格区分服务器状态(TanStack Query)与客户端状态(Zustand)；
- 陷阱：any 类型逃逸，失去 TS 价值。规避：CI 中加入 tsc --noEmit 检查。

Lab 6 核心特性端到端开发与 Sprint Review（2 学时，第 11 周）

学时	2 学时	周次	第 11 周	Sprint	Sprint 3	地点	实验室

【实验目标】

- 完成 Sprint 3 计划承诺的核心功能开发：投票、采纳、评论、标签、声誉计算；
- 体验真实的 Sprint Review 流程：演示、收集反馈、纳入下个 Sprint；
- 验证多模块协作：身份认证模块通过领域事件影响声誉模块；
- 排查并修复 1-2 个跨模块协作 BUG，沉淀经验。

【环境与前置准备】

- Sprint 2 已正式结案，代码合并到 main 并通过 CI
- 团队已完成 Sprint 3 Planning，Sprint Backlog 已锁定
- 可演示的稳定环境：本地或测试服务器

【实验步骤】

Step 1 编码冲刺(60 分钟)

- 严格遵守 Sprint Backlog，不接受范围蔓延
- 所有新代码必须先写单元测试
- 每完成一个 Task 即提交 PR、走评审、合并
- 教师巡回评审，对关键设计点现场指导

Step 2 Sprint Review 演示(35 分钟，每队 5 分钟)

- PO 简要回顾本 Sprint 目标
- Developer 现场演示新功能

- ○ "利益相关者"(其他团队 + 教师)提问并打分
- ○ PO 决定哪些反馈进入 Sprint 4 Backlog

Step 3 Sprint Retrospective(25 分钟)

- ○ 使用"Start / Stop / Continue"或"Sailboat"模板
- ○ 每位成员匿名贴 3 张卡片
- ○ 团队投票选出 2-3 项改进措施纳入下个 Sprint
- ○ 提交 docs/retro/sprint-3.md 回顾纪要

【提交物】

1. Sprint 3 增量在 main 分支可演示；

2. Sprint Review 演示视频或现场录屏；

3. Sprint Retrospective 回顾纪要 Markdown；

4. 本 Sprint 关键设计的 ADR 文档(至少 1 份)。

【实验评分要点】

- ● Sprint Backlog 完成率(承诺履行)(30%)；
- ● 演示中系统的稳定性与功能完整性(25%)；
- ● 回顾会的真实性与改进项的可执行性(20%)；
- ● 跨团队答辩中的应对质量(25%)。

Lab 7 测试、代码评审与质量门禁（2 学时，第 13 周）

学时	2 学时	周次	第 13 周	Sprint	Sprint 3	地点	实验室

【实验目标】

- ● 将团队单元测试覆盖率提升到 70% 以上，核心聚合达到 90%+；
- ● 完成至少 1 条契约测试与 1 条端到端测试；
- ● 搭建 SonarQube + JaCoCo + Checkstyle 质量门禁，接入 CI 流水线；
- ● 完成至少 4 次跨成员代码评审，沉淀团队评审清单。

【环境与前置准备】

- Sprint 3 核心代码已完成
- 本地 / 团队 Docker 中已部署 SonarQube
- 已学习 JaCoCo Maven 插件与 SonarQube Maven 插件用法

【实验步骤】

Step 1 单元测试集中编写(40 分钟)

- ○ 识别覆盖率盲点(JaCoCo 报告)
- ○ 优先补齐核心领域逻辑的测试
- ○ 使用 ArgumentCaptor 验证副作用调用
- ○ 清理"无效断言"测试用例

Step 2 集成测试与契约测试(25 分钟)

- ○ 基于 Testcontainers 编写 1-2 个完整集成测试(覆盖 Controller → Service → Repository)
- ○ 使用 Pact 编写消费者驱动契约测试(可选，作为加分项)

Step 3 质量门禁与 SonarQube 接入(20 分钟)

- ○ 在 pom.xml 配置 jacoco-maven-plugin，生成 jacoco.xml 报告
- ○ 配置 sonar-project.properties，运行 mvn sonar：sonar 推送报告
- ○ 在 SonarQube 配置质量门禁：Coverage > 70%、Bugs = 0、Vulnerabilities = 0
- ○ 将 Sonar 检查作为 CI 流水线必经步骤

Step 4 代码评审实战(35 分钟)

- ○ 每位成员从本 Sprint 选择 1 个 PR(自己未参与的)进行深度评审
- ○ 使用课程提供的评审清单(正确性、可读性、可测性、安全、性能、风格)
- ○ 在 PR 评论中至少留下 5 条实质性意见
- ○ 总结团队评审清单 v1.0，提交 docs/team/review-checklist.md

【提交物】

1. 团队 SonarQube Dashboard 截图(覆盖率、Bug、Smell 等)；

2. 至少 4 份高质量 PR 评审记录(可导出 PDF)；

3. 团队代码评审清单 v1.0；

4. JaCoCo 覆盖率报告 HTML。

【实验评分要点】

- 覆盖率达成情况(30%)；
- 质量门禁配置与持续生效(25%)；
- 评审意见质量(深度、可操作性)(25%)；
- 评审清单的团队适用性与具体性(20%)。

Lab 8 CI/CD 流水线、容器化部署与上线（2 学时，第 15 周）

学时	2 学时	周次	第 15 周	Sprint	Sprint 4	地点	实验室

【实验目标】

- 完成完整的 CI/CD 流水线：从 push 到自动部署到测试环境；
- 完成 Docker 镜像多阶段构建，docker-compose 一键启动整套环境；
- 完成 Prometheus + Grafana 监控接入，至少 5 个核心指标可观测；
- 系统正式"上线"到课程统一部署环境，可在校园网内通过域名访问。

【环境与前置准备】

- 已完成 L14、L15 理论学习
- 团队已申请课程提供的测试服务器子域名
- 已配置 GitHub / Gitee Secrets

【实验步骤】

Step 1 后端 Docker 镜像构建(20 分钟)

○ 编写多阶段 Dockerfile：builder(JDK + Maven)→ runtime(JRE only)

- ○ 镜像体积优化：启用 jlink 自定义 JRE / 使用 distroless 基础镜像
- ○ 以非 root 用户运行容器
- ○ 本地构建并验证启动

Step 2 前端镜像与 Nginx 配置(15 分钟)

- ○ 前端多阶段构建：builder(node)→ runtime(nginx：alpine)
- ○ 配置 Nginx 反向代理：/api → 后端服务
- ○ 前端 SPA 路由 fallback 配置

Step 3 docker-compose 编排(20 分钟)

- ○ 完整 compose：app + MySQL + Redis + nginx + Prometheus + Grafana
- ○ 健康检查与依赖顺序(depends_on with condition： service_healthy)
- ○ .env 文件管理环境变量，在 CI 中通过 Secrets 注入

Step 4 CI/CD 流水线扩展(30 分钟)

- ○ 在 build + test + sonar 之后增加：docker build → push to registry
- ○ 增加部署 Job：SSH 到测试服务器执行 docker-compose pull && up -d
- ○ Smoke Test：部署后自动调用 /actuator/health 验证

Step 5 监控配置与上线验证(15 分钟)

- ○ 部署后访问 Grafana Dashboard，确认指标采集正常
- ○ 配置至少 1 条告警规则(p95 > 500ms 或 5xx 错误率 > 1%)
- ○ 在团队 IM 群通告系统上线 URL，邀请其他团队"友测"

【提交物】

1. 可访问的线上系统 URL(校园网内或公网)；

2. CI/CD 流水线运行成功记录截图；

3. Grafana Dashboard 截图(含真实流量数据)；

4. 《运维手册 v1.0》：启动、停止、回滚、常见故障处置流程；

5. docker-compose.yml、Dockerfile、CI 配置等全部提交至仓库。

【实验评分要点】

- 流水线完整性与自动化程度(30%)；
- 部署成功率与系统稳定性(25%)；
- 镜像优化(体积、安全)与编排合理性(20%)；
- 监控接入完整性与告警可用性(15%)；
- 运维手册可用性(10%)。

【常见陷阱与排查建议】

- 陷阱：在 Dockerfile 中硬编码数据库密码。规避：通过环境变量与 Secrets；
- 陷阱：容器跨主机网络不通。规避：统一在同一 Docker network 内编排；
- 陷阱：部署后访问 404。规避：Nginx 与 React Router 的 fallback 配置先在本地验证。